

安全加固报告 FIX-01：弱口令与密码爆破

修复漏洞类型：弱口令与密码爆破

修复小组：天亮了么

修复时间：2026-08-20

对应漏洞报告：VULN-01-弱口令与密码爆破

修复负责人：蒲云楷

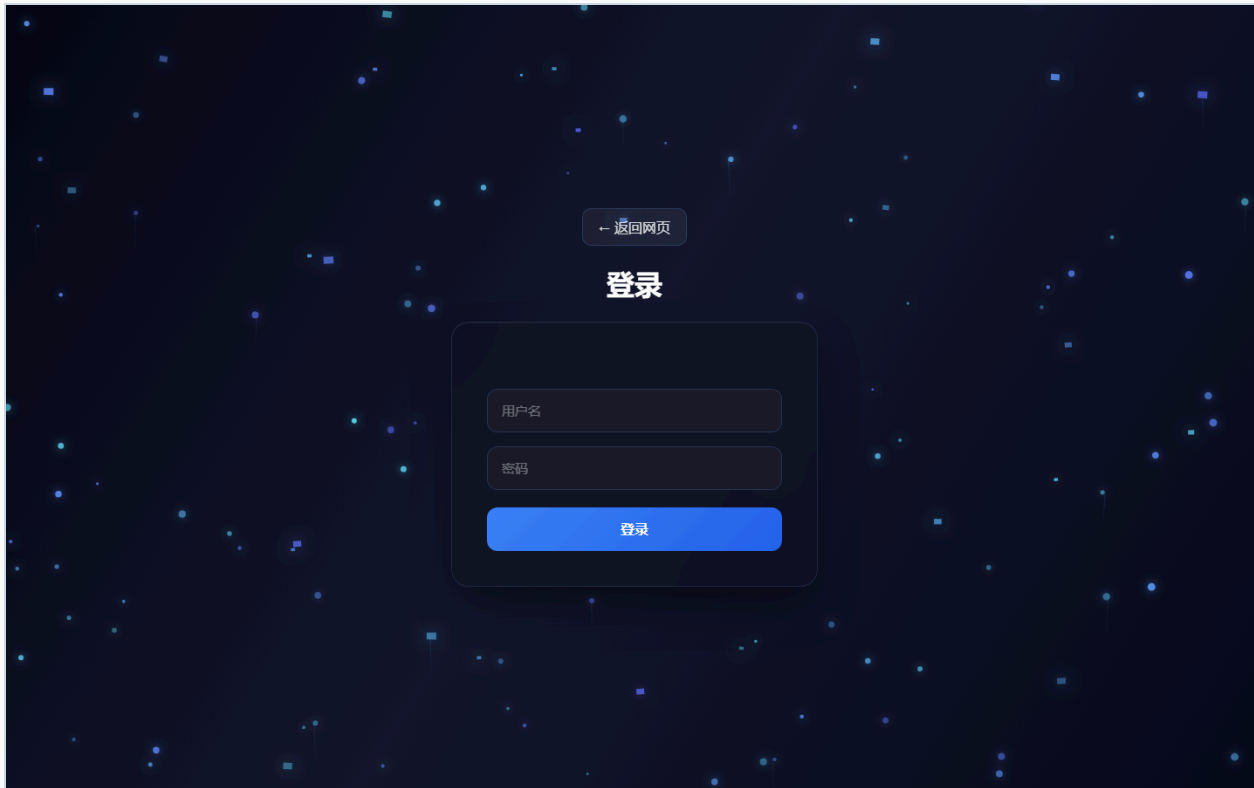
1. 基本信息

本报告按附录A记录阶段二任务1的修复和验证。

2. 漏洞复现情况

是否能按攻击方步骤复现：修复前基线已记录；修复后原始参数应被拦截或不产生越权效果。

复现环境：本机 127.0.0.1:5001。



[← 返回网页](#)

注册

密码长度至少为8位

weak-proof

密码

确认密码

注册

[← 返回网页](#)

登录

用户名或密码错误

pyknb

密码

登录

[← 返回网页](#)

登录

用户名或密码错误

dio

密码

登录

修复代码截图（实际源文件与行号）

任务 01 修复代码证据

gets.py:105-125

```
105 state = login_failures.setdefault(key, {'count': 0, 'locked_until': 0})
106 if state['locked_until'] and state['locked_until'] < time.time():
107     state['count'] = 0
108     state['locked_until'] = 0
109     state['count'] += 1
110 if state['count'] > LOGIN_MAX_FAILURES:
111     state['locked_until'] = time.time() + LOGIN_LOCK_SECONDS
112     security_logger.warning("login_failed username=%s ip=%s reason=%s count=%s, username, client_ip(), reason, state['count'])
113
114
115 def login_locked(username):
116     state = login_failures.get(login_key(username))
117     return bool(state and state['locked_until'] > time.time())
118
119
120 def clear_login_failures(username):
121     login_failures.pop(login_key(username), None)
122
123
124 def get_db_connection():
125     return pymysql.connect(**db_config)
```

gets.py:163-215

```
163 def login_page():
164     csrf_token()
165     return render_template("login.html")
166
167
168 @app.post("/login")
169 @csrf_protected
170 def login_post():
171     username = request.form.get("username", "").strip()
172     password = request.form.get("password", "")
173     result = _login(username, password, as_api=False)
174     if isinstance(result, tuple) and len(result) == 2 and isinstance(result[1], int) and result[1] >= 400:
175         return render_template("login.html", error=result[0], username=username, result[1])
176     return result
177
178
179 def _login(username, password, as_api=True):
180     if not username or not password:
181         response = {"error": "请输入用户名和密码"}
182         return jsonify(response), 400 if as_api else ("请输入用户名和密码", 400)
183     if login_locked(username):
184         response = {"error": "用户名或密码错误"}
185         return jsonify(response), 401 if as_api else ("用户名或密码错误", 401)
186     conn = get_db_connection()
187     try:
188         with conn.cursor() as cursor:
189             cursor.execute("SELECT username, password FROM users WHERE username=%s", (username,))
190             user = cursor.fetchone()
191         finally:
192             conn.close()
193     valid = bool(user) and (
194         check_password_hash(user[1], password)
195         if user and user[1].startswith("{pbkdf2}", "bcrypt:"))
196     else bool(user) and user[1] == password
197 )
198 if not valid:
199     record_login_failure(username)
200     response = {"error": "用户名或密码错误"}
201     return jsonify(response), 401 if as_api else ("用户名或密码错误", 401)
202 clear_login_failures(username)
203 session.clear()
204 csrf_token()
205 session["username"] = username
206 if user and not user[1].startswith("{pbkdf2}", "bcrypt:"):
207     conn = get_db_connection()
208     try:
209         with conn.cursor() as cursor:
210             cursor.execute("UPDATE users SET password=%s WHERE username=%s", (generate_password_hash(password), username))
211         conn.commit()
212     finally:
213         conn.close()
214     security_logger.info("login_success username=%s ip=%s", username, client_ip())
215     if as_api:
```

gets.py:267-302

```
267 @csrf_protected
268 def register_post():
269     data = request.form
270     result = _register(data.get("username", "").strip(), data.get("password", ""), data.get("confirm_password", ""))
271     if result[1] != 200:
272         payload = result[0].get_json(silent=True) if hasattr(result[0], "get_json") else 0
273         return render_template("register.html", error=payload or 0).get("error", "注册失败", username=data.get("username", "")), result[1]
274     return redirect("/")
275
276
277 def _register(username, password, confirm_password):
278     if not username or not password:
279         return jsonify({"error": "用户名和密码不能为空"}), 400
280     if password != confirm_password:
281         return jsonify({"error": "两次密码不一致"}), 400
282     policy_error = password_error(username, password)
283     if policy_error:
284         return jsonify({"error": policy_error}), 400
285     conn = get_db_connection()
286     try:
287         with conn.cursor() as cursor:
288             cursor.execute("SELECT 1 FROM users WHERE username=%s", (username,))
289             if cursor.fetchone():
290                 return jsonify({"error": "用户名已存在"}), 409
291             cursor.execute(
292                 "INSERT INTO users(username, password) VALUES(%s, %s)",
293                 (username, generate_password_hash(password)),
294             )
295         conn.commit()
296     finally:
297         conn.close()
298     return jsonify({"success": True}), 200
299
300
301 @app.post("/api/register")
302 @csrf_protected
```

3. 根因定位

相关文件：gets.py、相关模板和前端 API 客户端。

相关函数或接口：/login、/register、/api/login、/api/register。

问题代码/根因：gets.py 的 _login、_register 缺少密码策略、失败计数和安全日志。

4. 修复措施

修改点1

修改文件：gets.py。

修改前逻辑：缺少统一安全边界或校验。

修改后逻辑：增加8位及字母数字密码策略、弱口令黑名单、5次/5分钟锁定、统一错误提示、登录日志；历史明文密码成功登录后升级哈希。

修改点2

修改文件：相关 HTML/CSS/前端 API（按任务适用）。

修改后逻辑：页面和客户端与服务端安全校验保持一致，不能替代服务端校验。

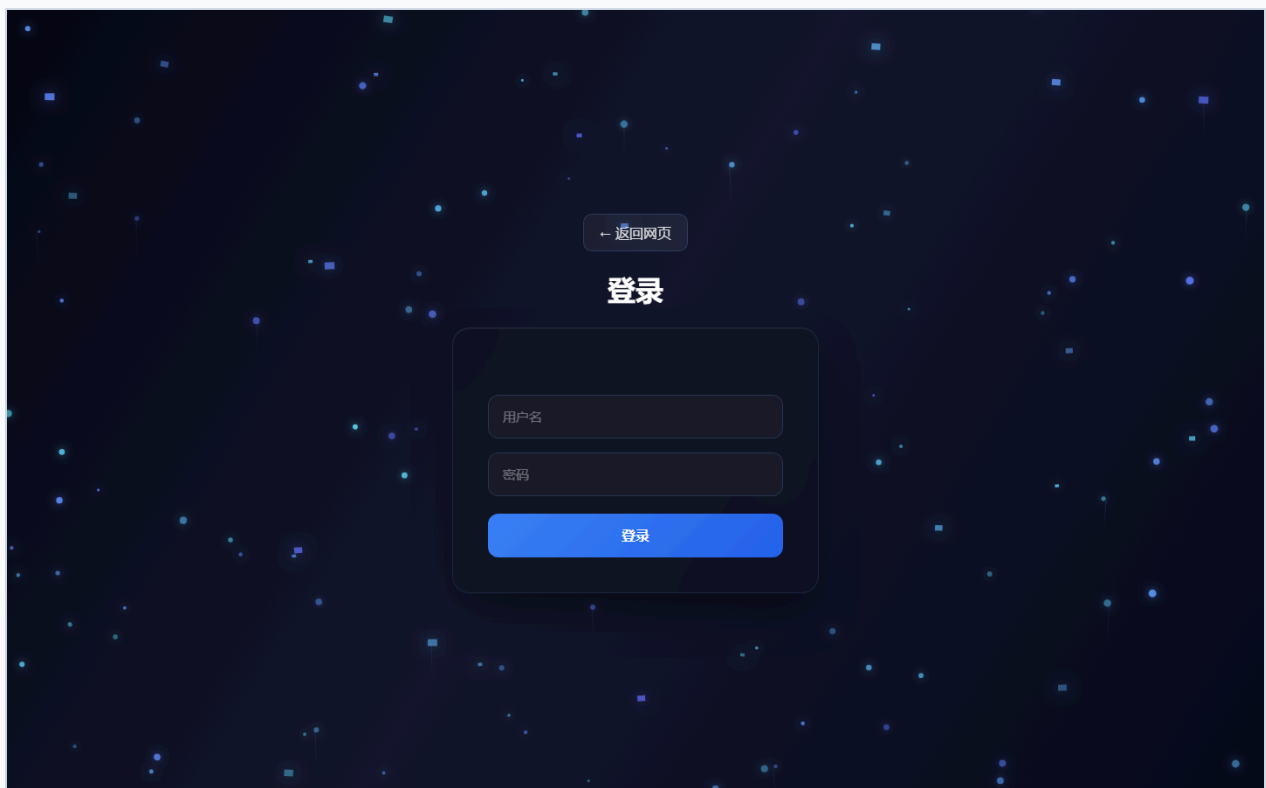
5. 修复后验证

使用攻击方原始步骤重新测试：

1. 发送原始无害测试参数。
2. 观察状态码、页面、数据、文件和日志。
3. 确认原攻击失效并保存截图。

验证结果：历史账号存在 dio/dio、zxrnb/123456 等弱密码；修复前可直接登录。修复后连续5次错误返回401并锁定，弱密码注册返回400。

原攻击是否失效：是（本机回归测试）



← 返回网页

注册

密码长度至少为8位

weak-proof

密码

确认密码

注册

← 返回网页

登录

用户名或密码错误

pyknb

密码

登录

[← 返回网页](#)

登录

用户名或密码错误

dio

密码

登录

修复代码截图（实际源文件与行号）

任务 01 修复代码证据

gets.py:105-125

```
105 state = login_failures.setdefault(key, {'count': 0, 'locked_until': 0})
106 if state['locked_until'] and state['locked_until'] < time.time():
107     state['count'] = 0
108     state['locked_until'] = 0
109     state['count'] += 1
110 if state['count'] > LOGIN_MAX_FAILURES:
111     state['locked_until'] = time.time() + LOGIN_LOCK_SECONDS
112     security_logger.warning("login_failed username=%s ip=%s reason=%s count=%s", username, client_ip(), reason, state['count'])
113
114
115 def login_locked(username):
116     state = login_failures.get(login_key(username))
117     return bool(state and state['locked_until'] > time.time())
118
119
120 def clear_login_failures(username):
121     login_failures.pop(login_key(username), None)
122
123
124 def get_db_connection():
125     return pymysql.connect(**db_config)
```

gets.py:163-215

```
163 def login_page():
164     csrf_token()
165     return render_template("login.html")
166
167
168 @app.post("/login")
169 @csrf_protected
170 def login_post():
171     username = request.form.get("username", "").strip()
172     password = request.form.get("password", "")
173     result = _login(username, password, as_api=False)
174     if isinstance(result, tuple) and len(result) == 2 and isinstance(result[1], int) and result[1] >= 400:
175         return render_template("login.html", error=result[0], username=username, result[1])
176     return result
177
178
179 def _login(username, password, as_api=True):
180     if not username or not password:
181         response = {'error': "请输入用户名和密码"}
182         return jsonify(response), 400 if as_api else ("请输入用户名和密码", 400)
183     if login_locked(username):
184         response = {'error': "用户名或密码错误"}
185         return jsonify(response), 401 if as_api else ("用户名或密码错误", 401)
186     conn = get_db_connection()
187     try:
188         with conn.cursor() as cursor:
189             cursor.execute("SELECT username, password FROM users WHERE username=%s", (username,))
190             user = cursor.fetchone()
191         finally:
192             conn.close()
193     valid = bool(user) and (
194         check_password_hash(user[1], password)
195         if user and user[1].startswith("{pbkdf2}", "script:")
196         else bool(user) and user[1] == password
197     )
198     if not valid:
199         record_login_failure(username)
200         response = {'error': "用户名或密码错误"}
201         return jsonify(response), 401 if as_api else ("用户名或密码错误", 401)
202     clear_login_failures(username)
203     session.clear()
204     csrf_token()
205     session["username"] = username
206     if user and not user[1].startswith("{pbkdf2}", "script:"):
207         conn = get_db_connection()
208         try:
209             with conn.cursor() as cursor:
210                 cursor.execute("UPDATE users SET password=%s WHERE username=%s", (generate_password_hash(password), username))
211             conn.commit()
212         finally:
213             conn.close()
214     security_logger.info("login_success username=%s ip=%s", username, client_ip())
215     if as_api:
```

gets.py:267-302

```
267 @csrf_protected
268 def register_post():
269     data = request.form
270     result = _register(data.get("username", "").strip(), data.get("password", ""), data.get("confirm_password", ""))
271     if result[1] != 200:
272         payload = result[0].get_json(silent=True) if hasattr(result[0], "get_json") else 0
273         return render_template("register.html", error=payload or 0).get("error", "注册失败", username=data.get("username", "")), result[1]
274     return redirect("/")
275
276
277 def _register(username, password, confirm_password):
278     if not username or not password:
279         return jsonify({'error': "用户名和密码不能为空"}), 400
280     if password != confirm_password:
281         return jsonify({'error': "两次密码不一致"}), 400
282     policy_error = _password_error(username, password)
283     if policy_error:
284         return jsonify({'error': policy_error}), 400
285     conn = get_db_connection()
286     try:
287         with conn.cursor() as cursor:
288             cursor.execute("SELECT 1 FROM users WHERE username=%s", (username,))
289             if cursor.fetchone():
290                 return jsonify({'error': "用户名已存在"}), 409
291             cursor.execute(
292                 "INSERT INTO users(username, password) VALUES(%s, %s)",
293                 (username, generate_password_hash(password)),
294             )
295             conn.commit()
296         finally:
297             conn.close()
298     return jsonify({'success': True}), 200
299
300
301 @app.post("/api/register")
302 @csrf_protected
```

6. 额外加固

统一错误提示、输入长度限制、安全日志、Cookie 安全属性、调试模式关闭和生产密钥配置要求。

7. 仍需改进

课程组别、负责人、Git tag 等元数据需在提交前填写；HTTPS 部署时启用 Secure Cookie；Web 服务器需确认上传目录不可执行。